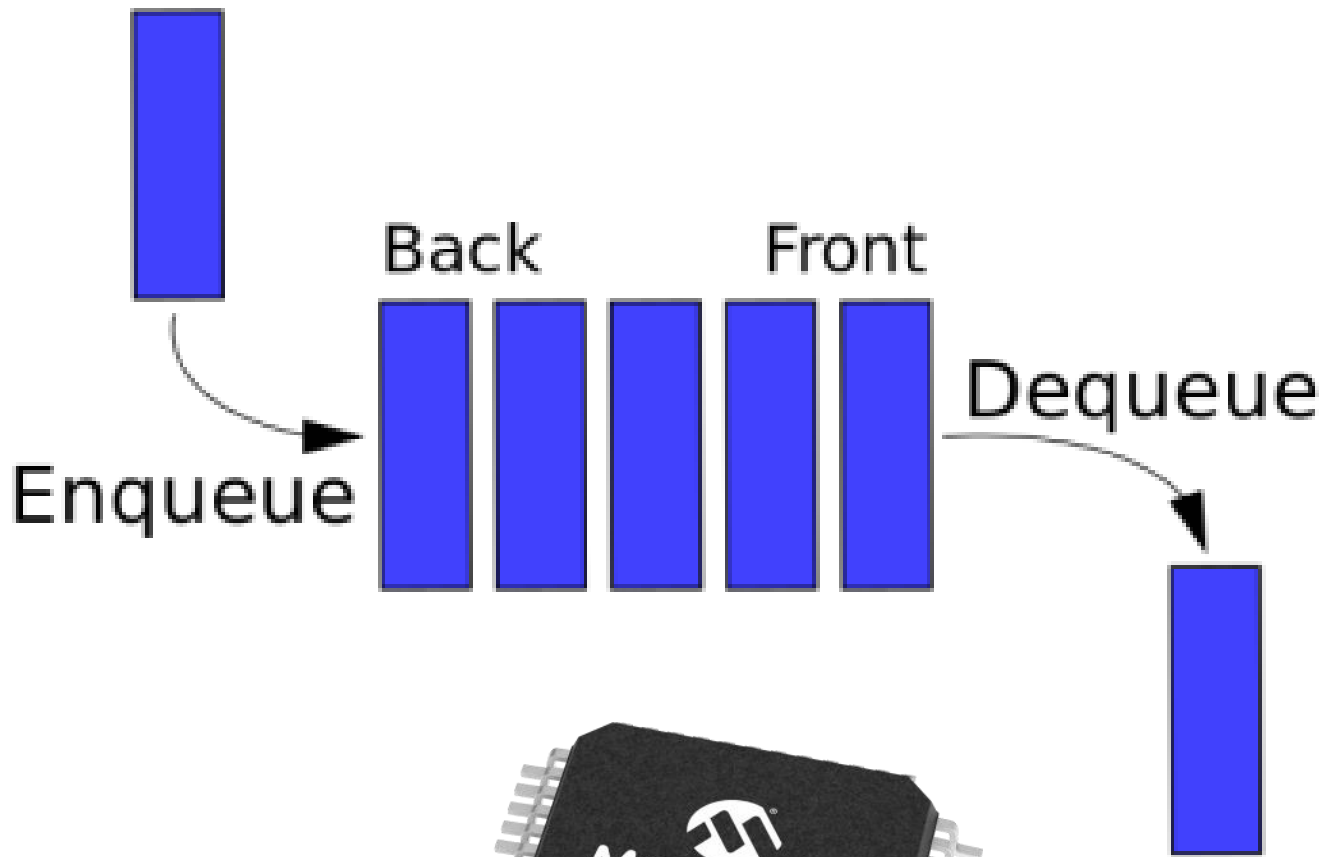


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 28 - Queues and Buffers in Embedded Systems



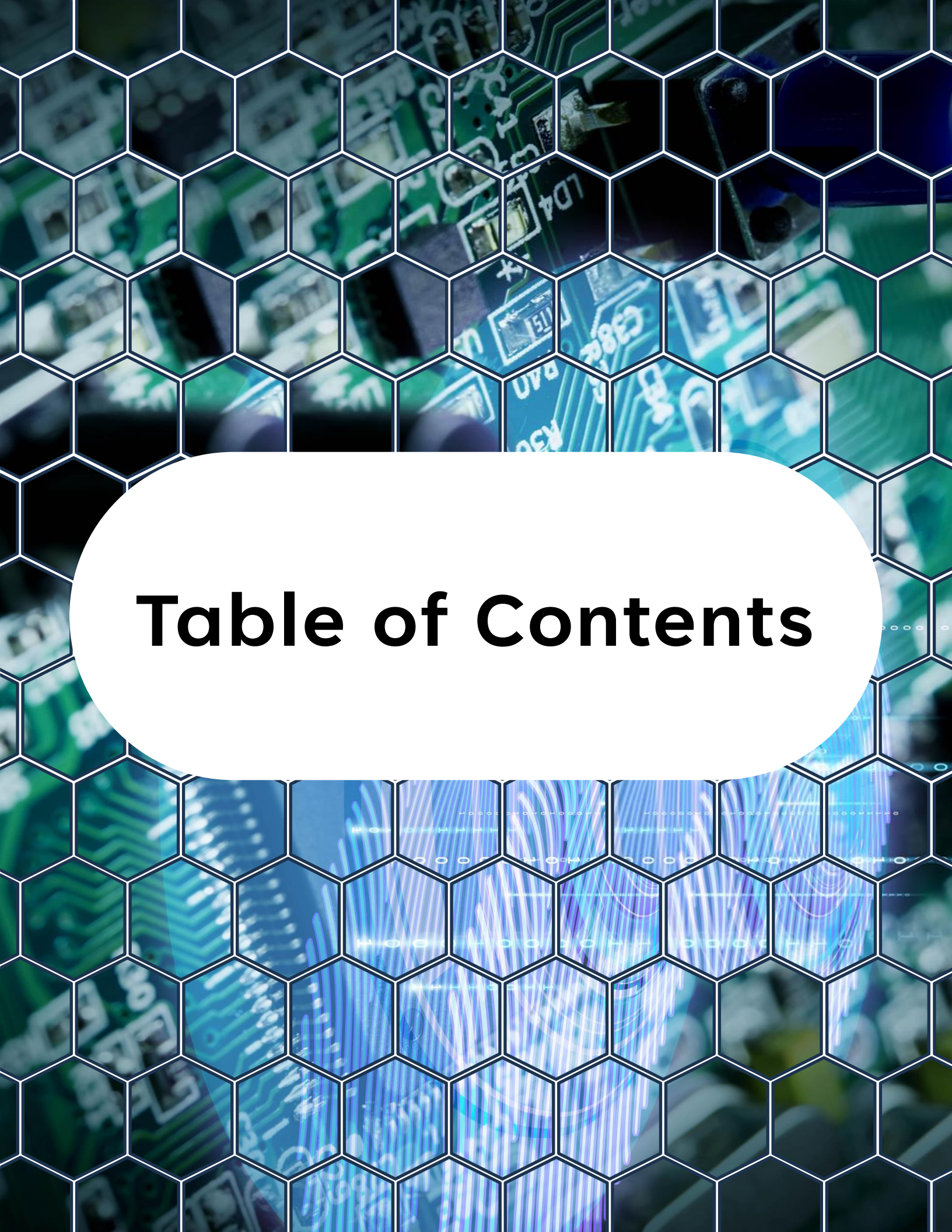
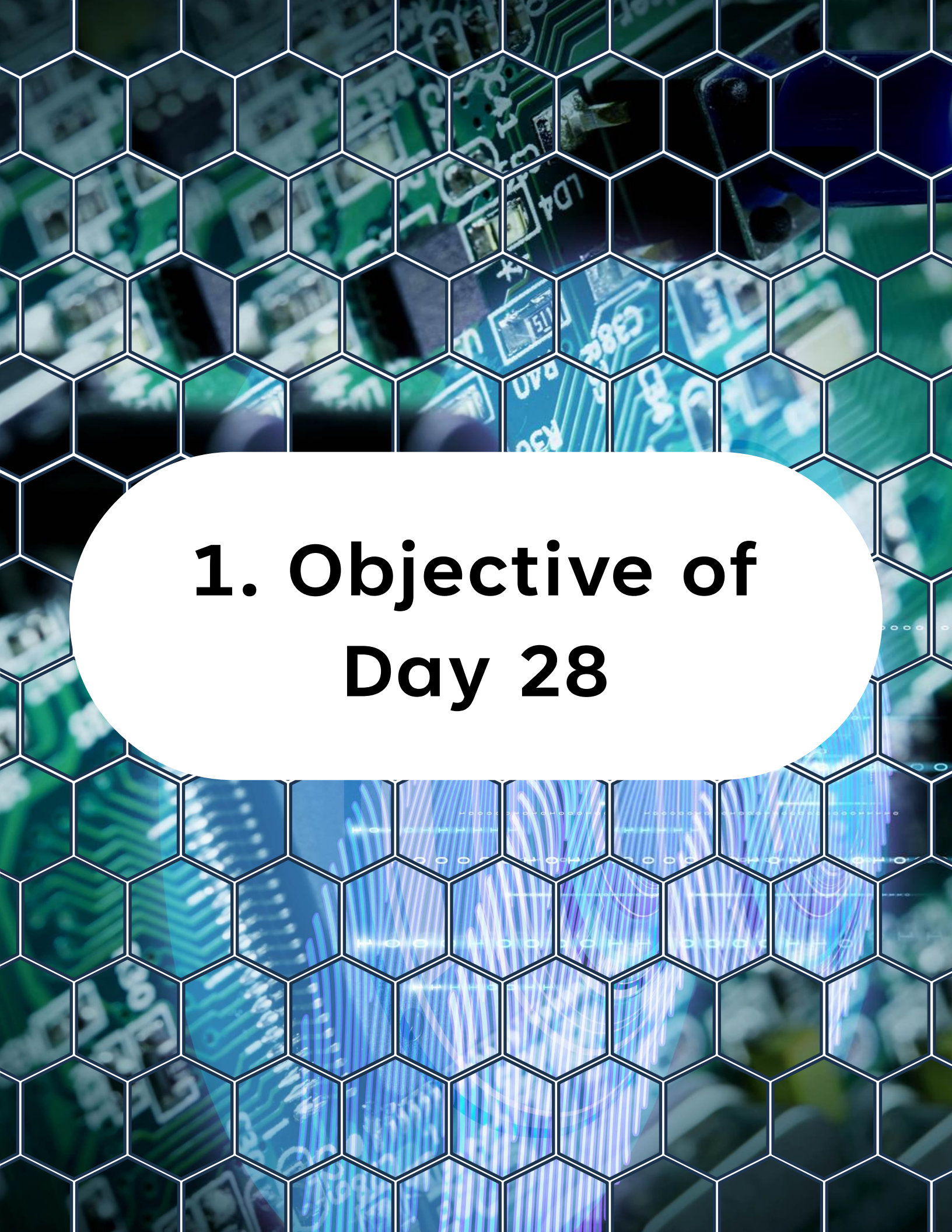


Table of Contents

Table of Contents

1. Objective of Day 28
2. The Core Problem
3. Producer-Consumer Model
4. What Is a FIFO Buffer?
5. Circular Buffer Concept
6. Basic FIFO Implementation
7. Decoupling Modules
8. Safe Data Movement Between Contexts
9. Lab - UART RX FIFO Buffer
10. What You Should Observe
11. Common Design Patterns
12. Common Mistakes
13. What You Should Understand Now
14. Preview of Day 29



1. Objective of Day 28

1. Objective of Day 28

As your firmware grows, different parts of the system start producing and consuming data at different rates.

Examples:

UART receives bytes asynchronously

ADC produces samples periodically

Application processes data at its own pace

If you connect these directly, you create fragile systems.

Today we introduce the solution:

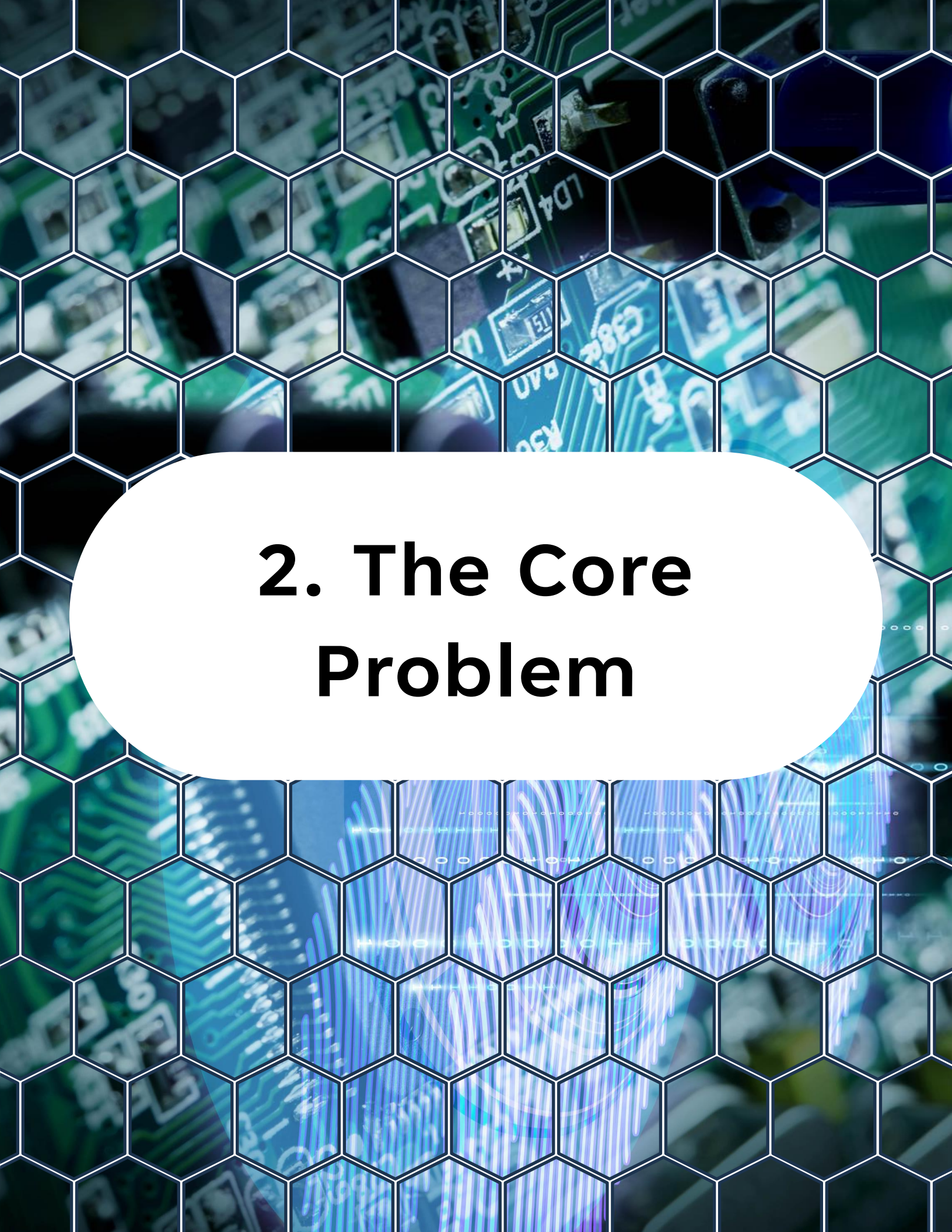
Queues and Buffers

1. Objective of Day 28

By the end of today, you will understand:

- Producer-consumer flow
- FIFO buffering
- How to decouple modules
- How to safely move data between ISR and main context

This is a key step toward building scalable embedded firmware.



2. The Core Problem

2. The Core Problem

Consider this situation:

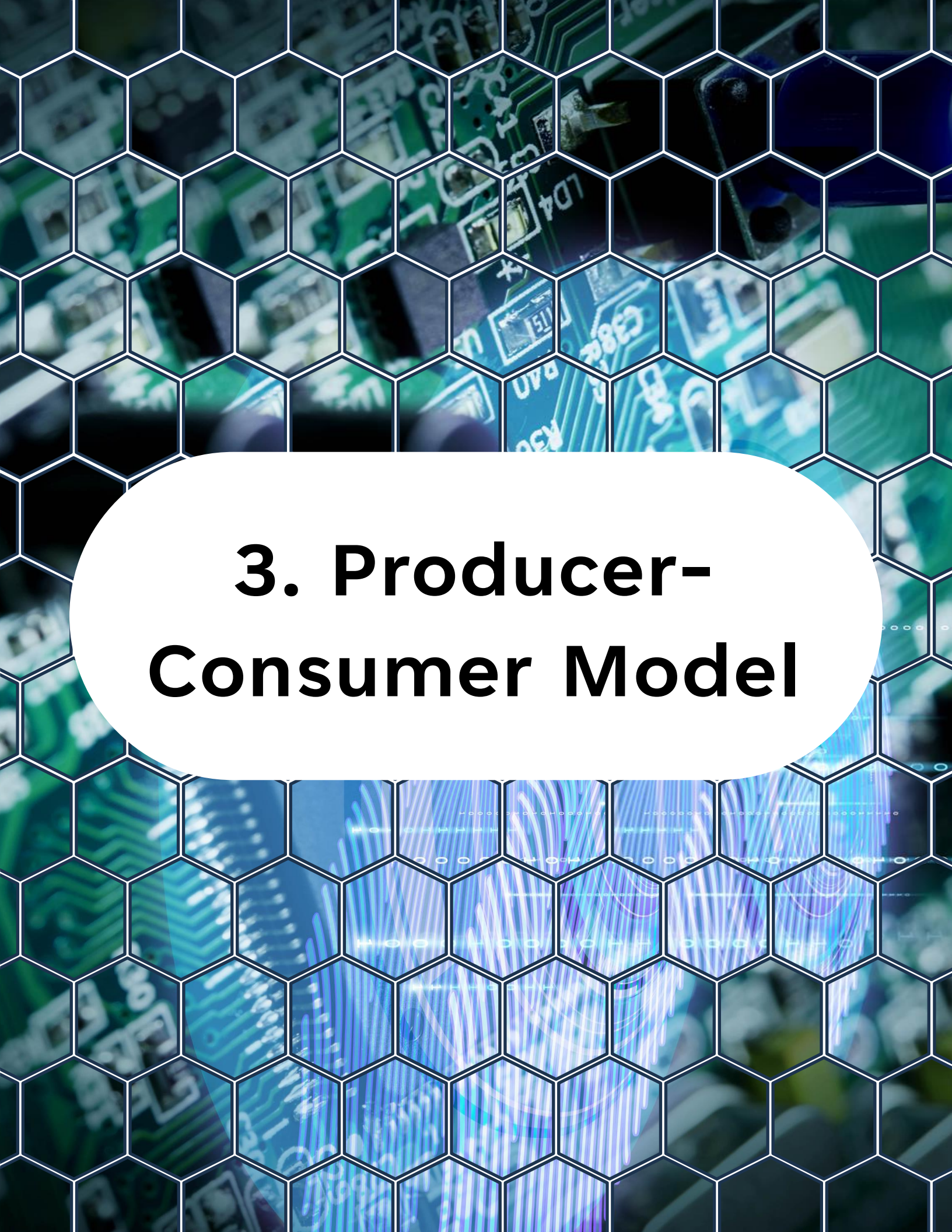
- UART receives data in an interrupt
- Main loop processes commands

If the main loop is busy:

Incoming data may be lost.

Why?

Because there is no buffering between producer and consumer.



3. Producer- Consumer Model

3. Producer-Consumer Model

This is one of the most important patterns in embedded systems.

- **Producer** generates data
- **Consumer** processes data

They operate independently.

Example

Producer:

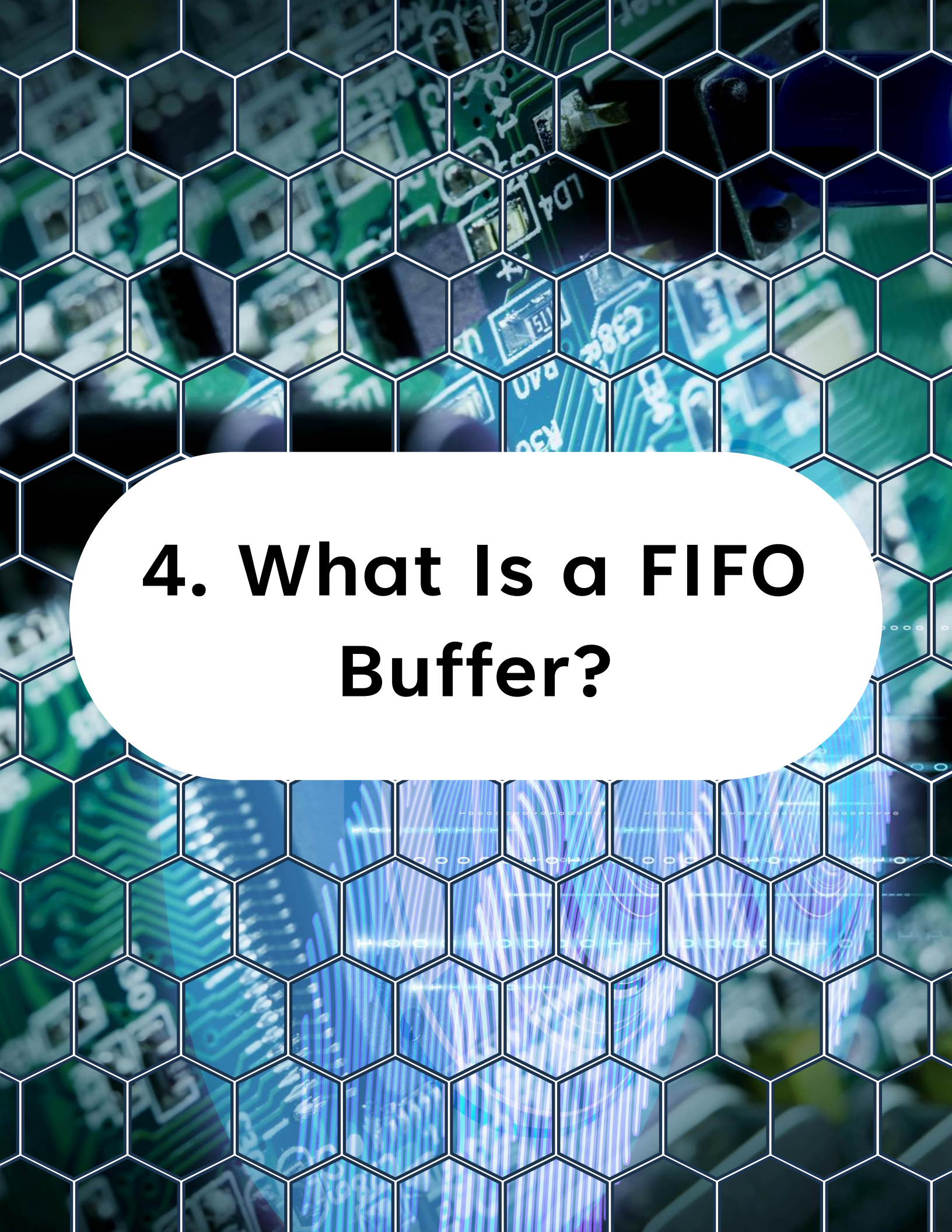
- UART ISR receives bytes

Consumer:

- Main loop parses commands

They run at different speeds.

Without a buffer, they interfere with each other.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', and 'U7A'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center of the slide, there is a large white oval containing the title text.

4. What Is a FIFO Buffer?

4. What Is a FIFO Buffer?

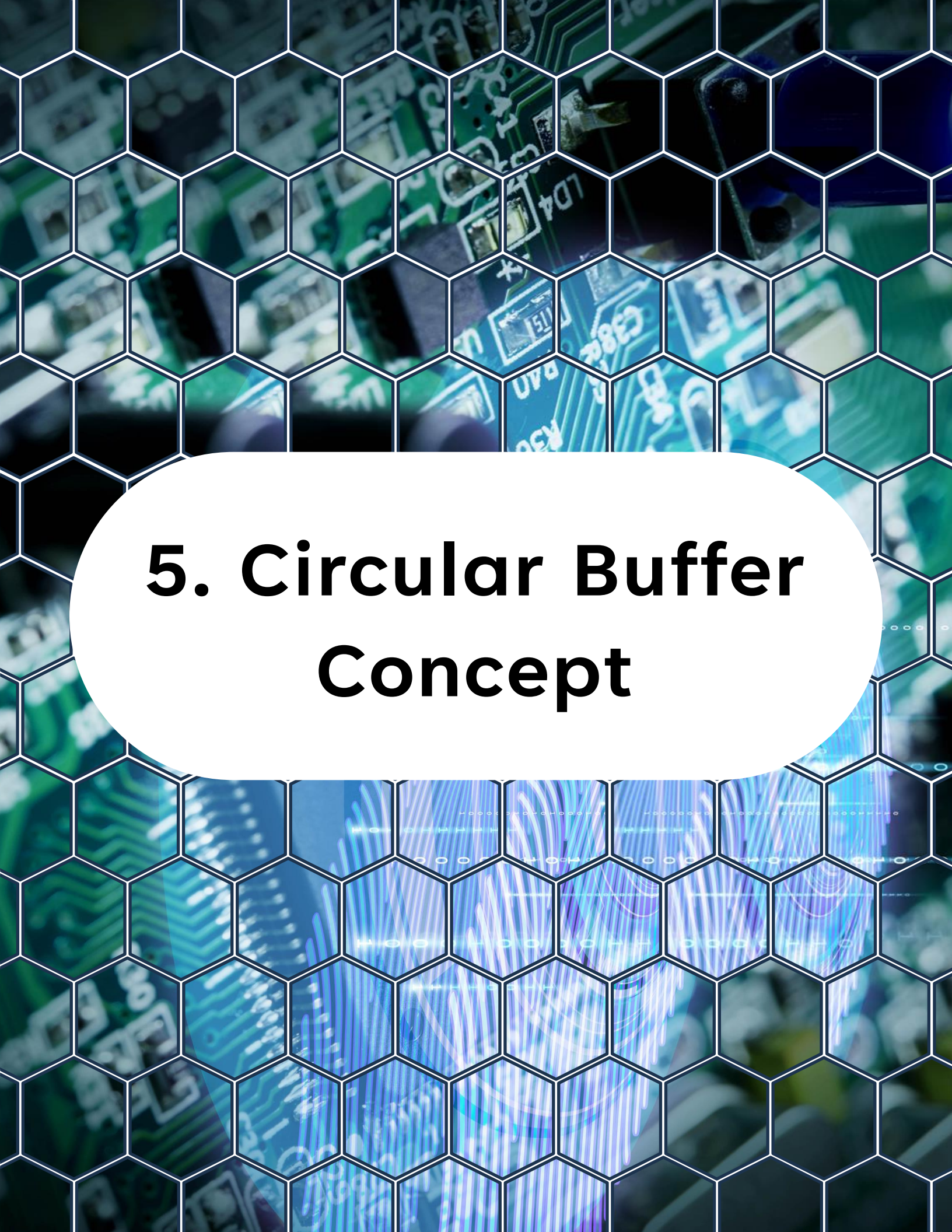
FIFO = First-In, First-Out

Data is read in the same order it was written.

This is the most common buffer type in embedded systems.

FIFO Properties

- Fixed size
- Circular memory
- Separate read and write indices
- Non-blocking operation



5. Circular Buffer Concept

5. Circular Buffer Concept

A FIFO is usually implemented as a circular buffer.

Memory layout:

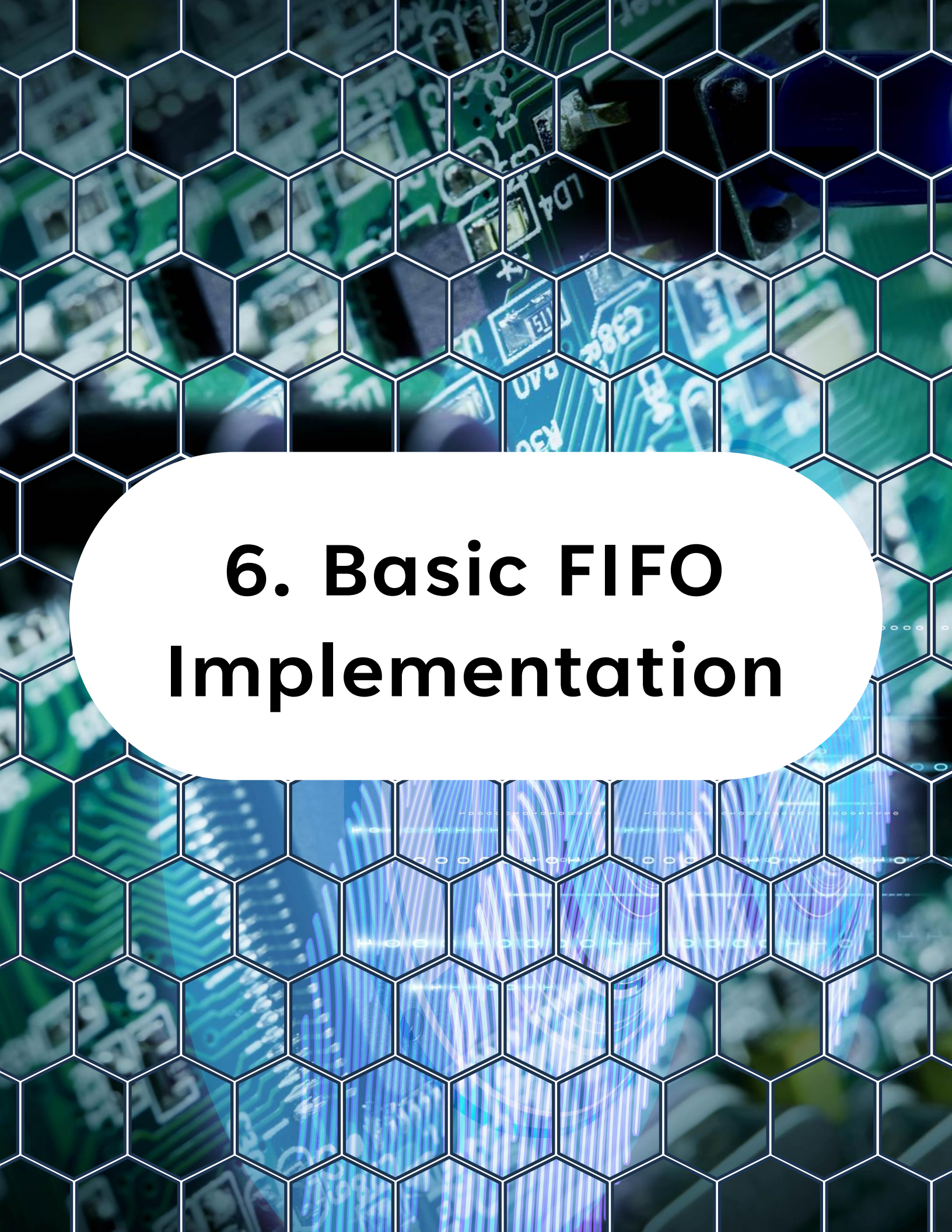
[0][1][2][3][4][5][6][7]

Two pointers:

- **Head** (write position)
- **Tail** (read position)

When reaching the end:

Wrap around to index 0.



6. Basic FIFO Implementation

6. Basic FIFO Implementation

6.1 Buffer Definition



```
1 // Define the size of the FIFO buffer
2 #define FIFO_SIZE 64
3
4 // Simple FIFO buffer implementation
5 typedef struct
6 {
7     uint8_t buffer[FIFO_SIZE];
8     volatile uint8_t head;
9     volatile uint8_t tail;
10 } fifo_t;
```

6.2 Initialization



```
1 // Initialize a FIFO buffer
2 static void fifo_init(fifo_t *f)
3 {
4     f->head = 0;
5     f->tail = 0;
6 }
```

6. Basic FIFO Implementation

6.3 Check Empty



```
1 // Check if the FIFO is empty
2 static uint8_t fifo_is_empty(fifo_t *f)
3 {
4     return (f->head == f->tail);
5 }
```

6.4 Check Full



```
1 // Check if the FIFO is full
2 static uint8_t fifo_is_full(fifo_t *f)
3 {
4     return ((f->head + 1) % FIFO_SIZE) == f->tail;
5 }
```


6. Basic FIFO Implementation

6.5 Push (Write)

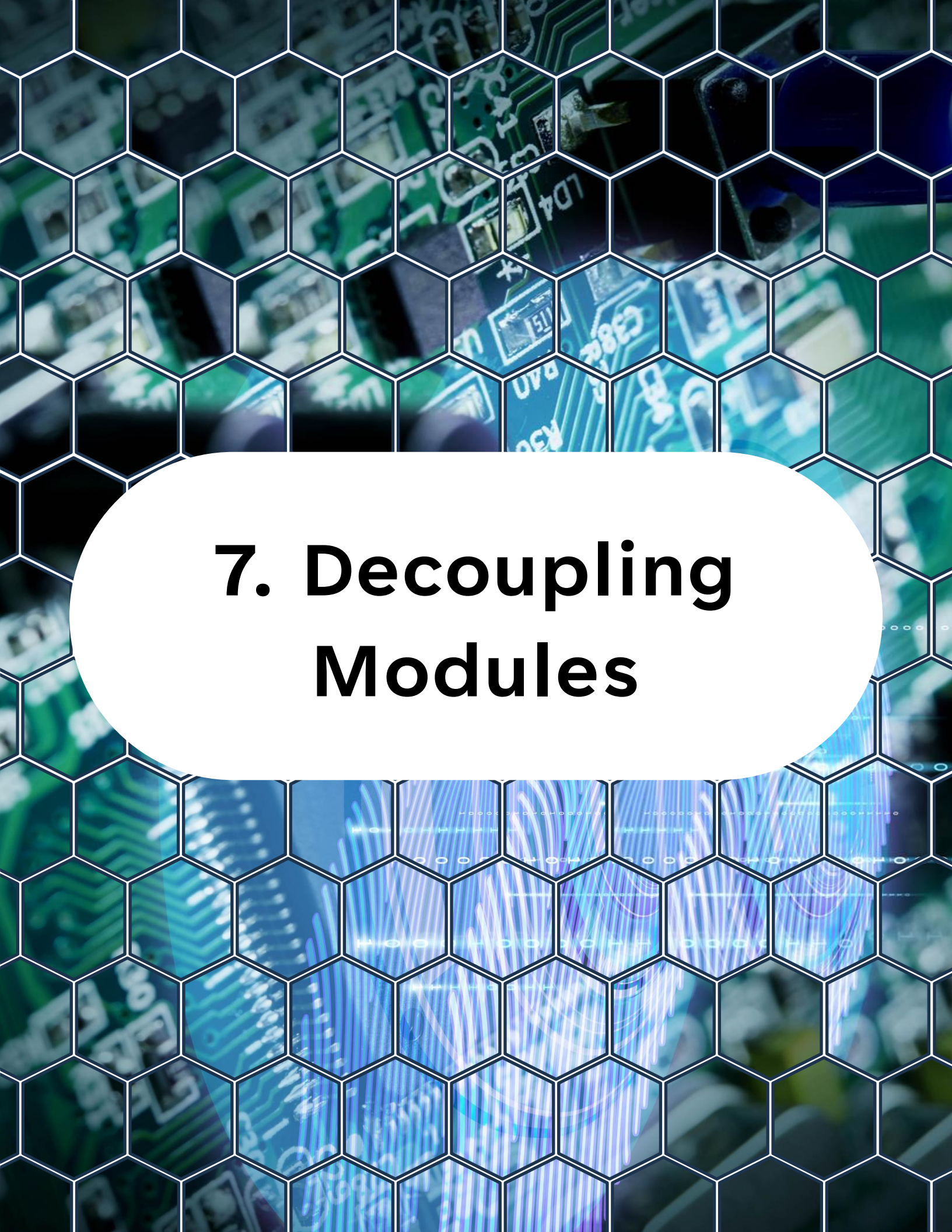
```
1  /**
2   * @brief Pushes a byte of data into the FIFO buffer.
3   *
4   * @param f Pointer to the FIFO buffer structure.
5   * @param data The byte of data to be pushed into the buffer.
6   * @return uint8_t Returns 1 if the data was successfully
7   *         pushed, or 0 if the buffer is full.
8   */
9  static uint8_t fifo_push(fifo_t *f, uint8_t data)
10 {
11     uint8_t next = (f->head + 1) % FIFO_SIZE;
12
13     if (next == f->tail)
14     {
15         return 0; /* Buffer full */
16     }
17
18     f->buffer[f->head] = data;
19     f->head = next;
20
21     return 1;
22 }
```

6. Basic FIFO Implementation

6.6 Pop (Read)



```
1  /**
2   * @brief Pops a byte of data from the FIFO buffer.
3   *
4   * @param f Pointer to the FIFO buffer structure.
5   * @param data Pointer to the location where the popped data
6   *             will be stored.
7   * @return uint8_t Returns 1 if data was successfully popped,
8   *             or 0 if the buffer is empty.
9   */
10 static uint8_t fifo_pop(fifo_t *f, uint8_t *data)
11 {
12     if (f->head == f->tail)
13     {
14         return 0; /* Buffer empty */
15     }
16
17     *data = f->buffer[f->tail];
18     f->tail = (f->tail + 1) % FIFO_SIZE;
19
20     return 1;
21 }
```

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', and 'U7A'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the section title in bold black text.

7. Decoupling Modules

7. Decoupling Modules

With a FIFO:

- Producer writes to buffer
- Consumer reads from buffer

They no longer depend on each other's timing.

Example

UART ISR:



```
1 // Interrupt Service Routine for USART0 Receive Complete
2 ISR(USART0_RXC_vect)
3 {
4     // Read the received data from the USART0 data register
5     uint8_t data = USART0.RXDATA;
6
7     // Push the received data into the UART RX FIFO buffer
8     fifo_push(&uart_rx_fifo, data);
9 }
```

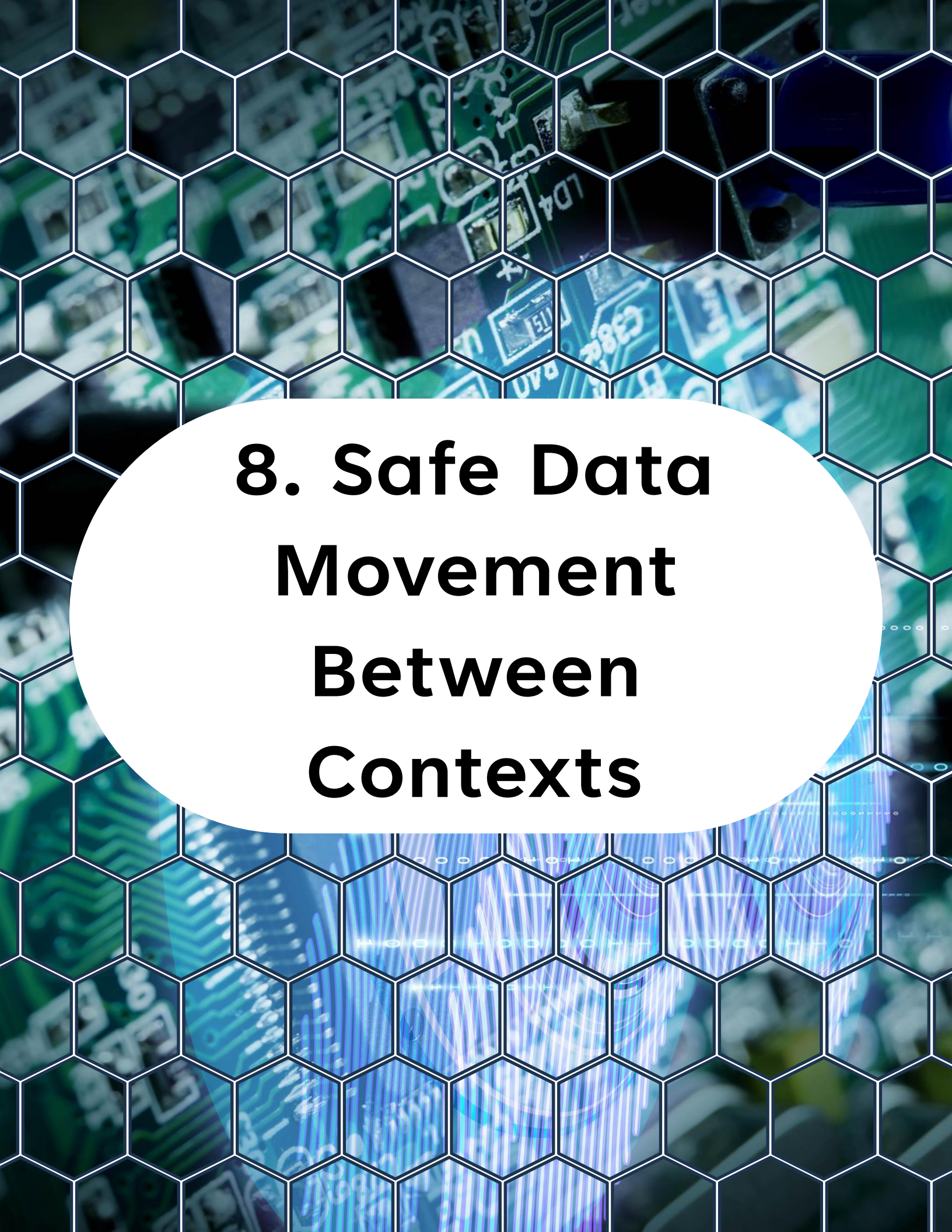
7. Decoupling Modules

Main loop:



```
1  uint8_t data;  
2  
3  // Check if there is data in the UART RX FIFO and process it  
4  if (fifo_pop(&uart_rx_fifo, &data))  
5  {  
6      process_byte(data);  
7  }
```

This is clean and scalable.



8. Safe Data Movement Between Contexts

8. Safe Data Movement Between Contexts

In embedded systems, data often crosses boundaries:

- ISR -> main loop
- Timer -> application
- Peripheral -> driver

This introduces concurrency concerns.

8.1 Why Volatile Matters

Shared variables must be declared **volatile**.



```
1 volatile uint8_t head;  
2 volatile uint8_t tail;
```

This prevents compiler optimization issues.

8. Safe Data Movement Between Contexts

8.2 Atomic Access

For small variables (8-bit on AVR):

- Access is atomic
- No special protection needed

For larger variables (16-bit or more):

- Consider disabling interrupts briefly
- Or design access carefully

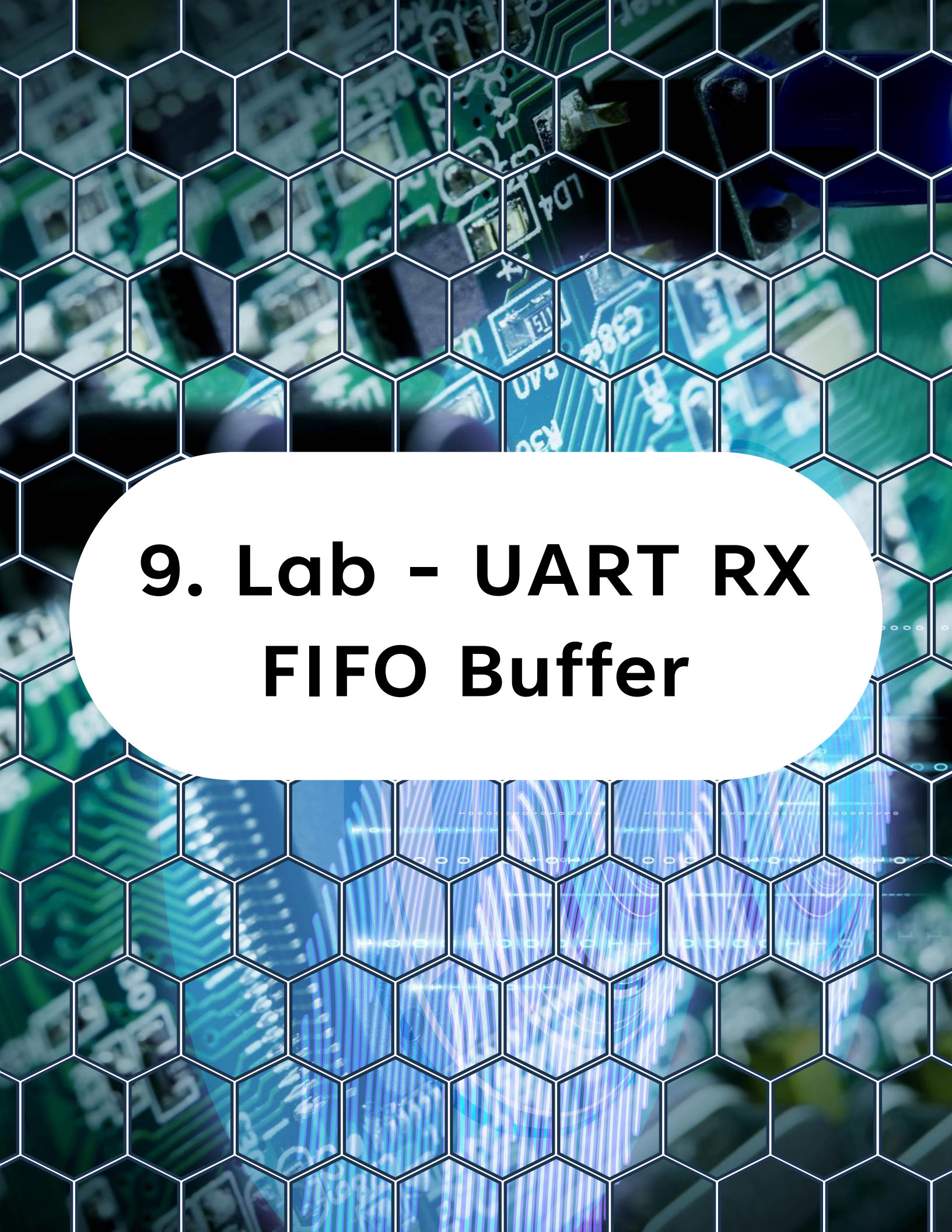
8.3 Avoid Complex Logic in ISR

ISR should only:

- Push data into buffer, and set flags

Never:

- Process entire messages
- Perform heavy computation



9. Lab - UART RX FIFO Buffer

9. Lab - UART RX FIFO Buffer

Goal

- Receive UART data using an interrupt
- Store data in a FIFO buffer
- Process data in main loop

FIFO Instance



```
1 static fifo_t uart_rx_fifo;
```

UART Initialization (Simplified)



```
1 // Initialize USART0 for 9600 bps, 8N1 configuration
2 static void USART0_init(void)
3 {
4     // Set baud rate to 9600 bps
5     USART0.BAUD = (uint16_t)USART0_BAUD_RATE(9600);
6
7     // Configure USART0: Asynchronous mode, 8 data bits,
8     // no parity, 1 stop bit
9     USART0.CTRLA = USART_CMODE_ASYNCHRONOUS_gc
10                  | USART_CHSIZE_8BIT_gc
```

9. Lab - UART RX FIFO Buffer

```
6
7 // Configure USART0: Asynchronous mode, 8 data bits,
8 // no parity, 1 stop bit
9 USART0.CTRLA = USART_CMODE_ASYNCHRONOUS_gc
10             | USART_CHSIZE_8BIT_gc
11             | USART_PMODE_DISABLED_gc
12             | USART_SBMODE_1BIT_gc;
13
14 // Enable receiver
15 USART0.CTRLB = USART_RXEN_bm;
16
17 // Enable RX Complete Interrupt
18 USART0.CTRLA = USART_RXCIE_bm;
19 }
```

ISR – Producer



```
1 // Interrupt Service Routine for USART0 Receive Complete
2 ISR(USART0_RXC_vect)
3 {
4     // Read the received data from the USART0 data register
5     uint8_t data = USART0.RXDATA;
6
7     // Push the received data into the UART receive FIFO buffer
8     fifo_push(&uart_rx_fifo, data);
9 }
```

9. Lab - UART RX FIFO Buffer

Main Loop - Consumer



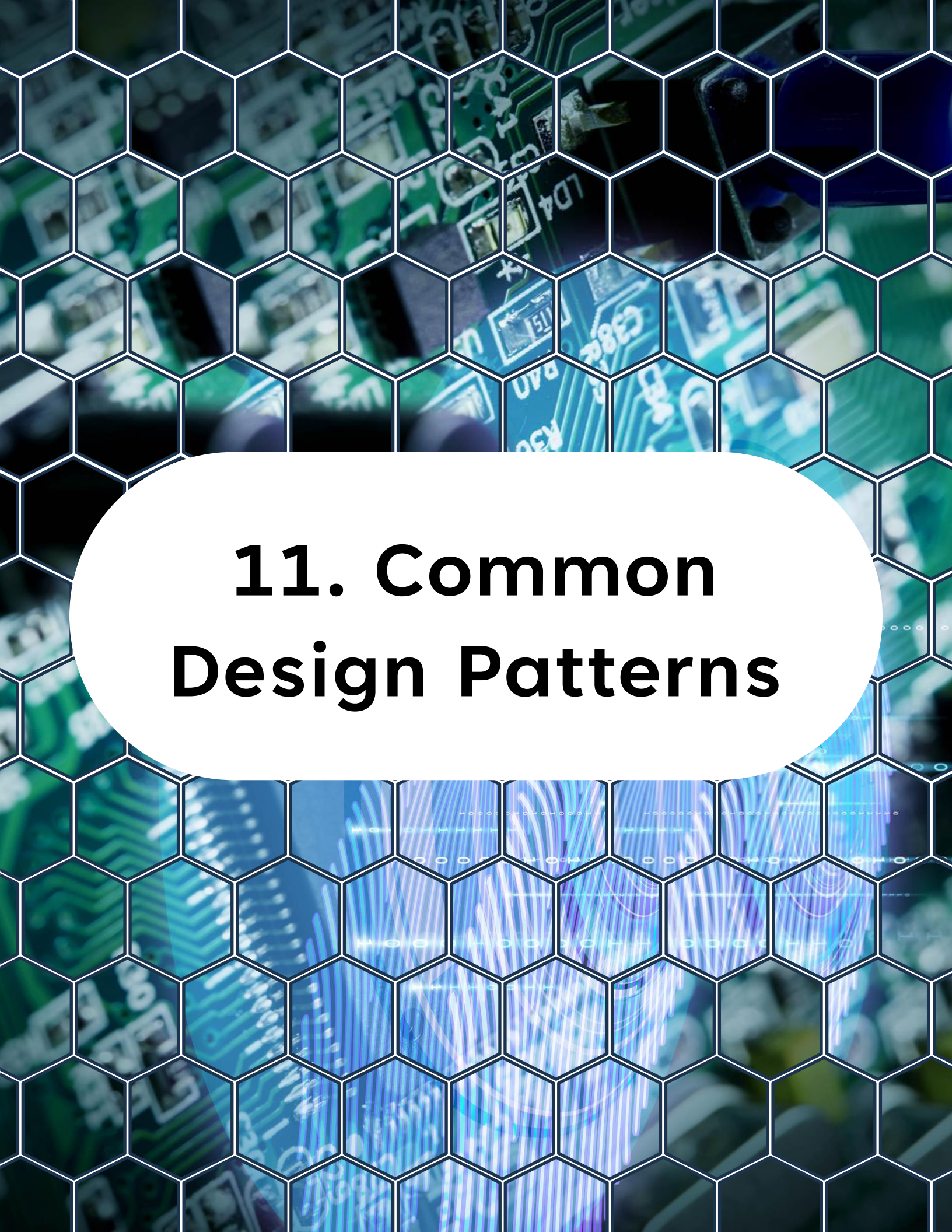
```
1  int main(void)
2  {
3      uint8_t data;
4
5      // Initialize the FIFO buffer for UART reception
6      fifo_init(&uart_rx_fifo);
7
8      // Initialize USART0 for serial communication
9      USART0_init();
10
11     // Enable global interrupts to allow USART0 RX
12     // interrupt to function
13     sei();
14
15     while (1)
16     {
17         // Check if there is data in the UART RX FIFO buffer
18         if (fifo_pop(&uart_rx_fifo, &data))
19         {
20             /* Process received byte */
21         }
22     }
23 }
```




10. What You Should Observe

10. What You Should Observe

- Data is received without loss (within buffer limits)
- ISR is minimal and fast
- Main loop processes data independently
- System remains responsive



11. Common Design Patterns

11. Common Design Patterns

11.1 Double Buffering

- One buffer is filled
- One buffer is processed

Used in:

- ADC sampling
- DMA-like behavior

11.2 Ring Buffers

Same as FIFO, widely used for:

- UART
- SPI
- Logging

11. Common Design Patterns

11.3 Message Queues

Instead of bytes:

- Pass structured data
- Example: commands, events



12. Common Mistakes

12. Common Mistakes

12.1 Not Checking Full Condition

Leads to buffer overflow and data corruption.

12.2 Blocking Inside Consumer

Defeats decoupling.

12.3 Large Buffers Without Need

Wastes RAM.

12.4 Ignoring Race Conditions

Especially with multi-byte variables.



13. What You Should Understand Now

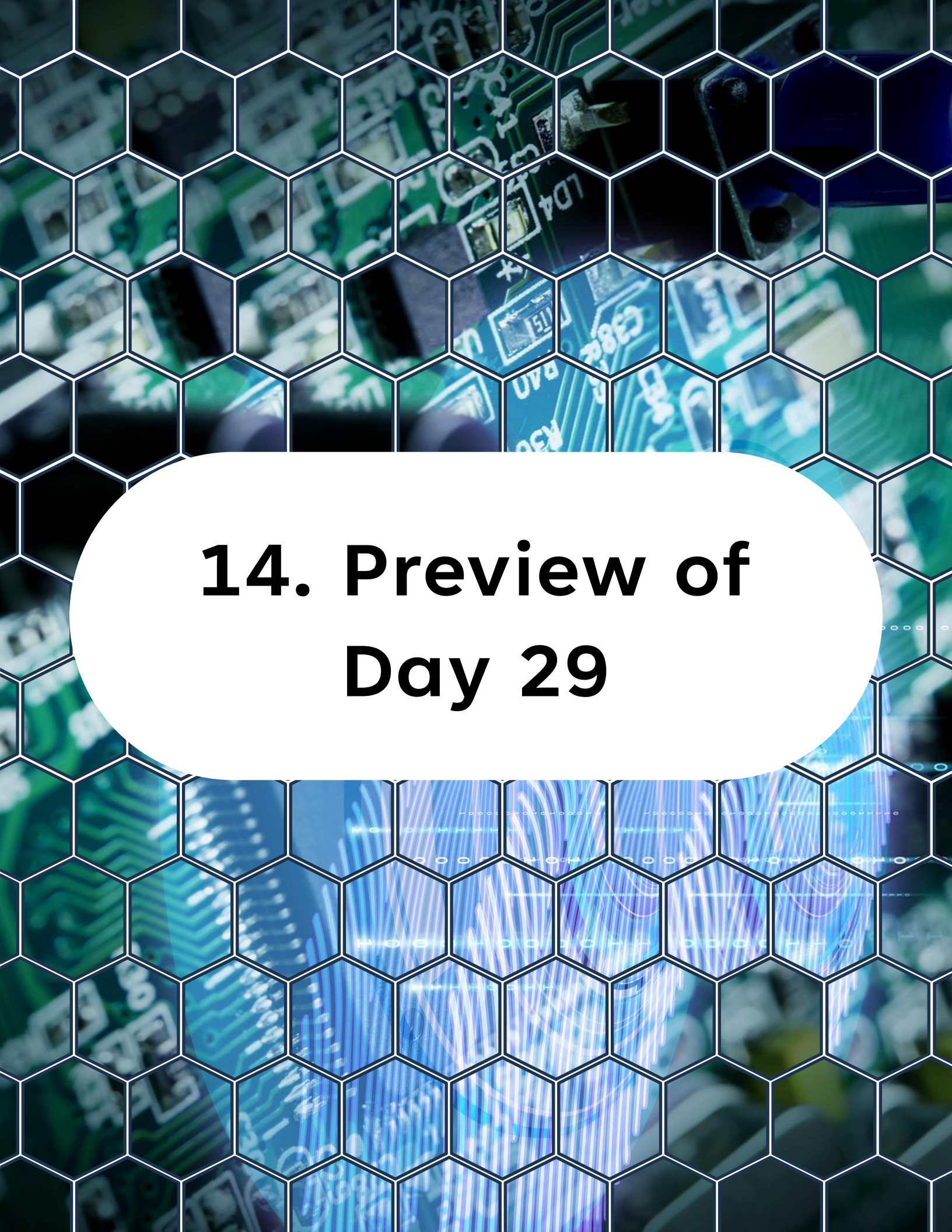
13. What You Should Understand Now

You now know:

- Producer-consumer architecture
- FIFO buffering
- How to decouple modules
- Safe data sharing between ISR and main
- How to design scalable communication paths

This is a fundamental building block for:

- Communication stacks
- Sensor pipelines
- Data logging systems



14. Preview of Day 29

14. Preview of Day 29

Next:

Cooperative Task Scheduling

We will explore:

- What a super-loop architecture is
- How cooperative scheduling works
- How to use timers for time slicing
- How state machines fit naturally into scheduled firmware
- How to design a clean non-blocking main loop

This is one of the most important steps between small examples and real embedded firmware.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>